WEEK-1

AIM: Image preprocessing techniques: resizing, filtering, thresholding using OpenCV

```
import cv2
import matplotlib.pyplot as plt
from google.colab import files
import numpy as np

# Upload image
uploaded = files.upload()

# Read uploaded image
filename = list(uploaded.keys())[0]
img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

# Resize
resized = cv2.resize(img, (200, 200))

# Gaussian Blur
blurred = cv2.GaussianBlur(resized, (5, 5), 0)

# Thresholding
_, thresholded = cv2.threshold(resized, 127, 255, cv2.THRESH_BINARY)

# Display
plt.figure(figsize=(12,4))

plt.subplot(131)
plt.imshow(resized, cmap='gray')
plt.title("Resized")
plt.axis('off')

plt.subplot(132)
plt.imshow(blurred, cmap='gray')
plt.title("Blurred")
plt.axis('off')

plt.subplot(133)
plt.imshow(thresholded, cmap='gray')
plt.title("Thresholded")
plt.axis('off')

plt.show()
```

WEEK-2

AIM: Edge detection using Sobel, Canny, and Laplacian operators

```
import cv2
import numpy as np
from google.colab import files
```

```python
from google.colab.patches import cv2_imshow

# Upload image
uploaded = files.upload()

# Read image
filename = list(uploaded.keys())[0]
img = cv2.imread(filename)

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Sobel Edge Detection
sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobelx, sobely)
sobel = cv2.convertScaleAbs(sobel)
# Canny Edge Detection
canny = cv2.Canny(gray, 100, 200)
# Laplacian Edge Detection
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)

# Display images
print("Original Image")
cv2_imshow(img)
```

```python
import numpy as np
from google.colab import files
from google.colab.patches import cv2_imshow

# Upload image
uploaded = files.upload()

# Read uploaded image
filename = list(uploaded.keys())[0]
img = cv2.imread(filename)

# Check if image is loaded
if img is None:
    print("Error: Unable to load image.")
else:
    # Convert to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # ---------------- HOG Feature Extraction ----------------
    hog = cv2.HOGDescriptor()
    hog_features = hog.compute(gray)

    print("HOG Features Shape:", hog_features.shape)

    # ---------------- SIFT Feature Extraction ----------------
    sift = cv2.SIFT_create()
```

WEEK-5

AIM:Implement face detection using Haar cascades or DNN models

```
import cv2
from google.colab import files
from google.colab.patches import cv2_imshow

# Load the Haar Cascade classifier
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

# Upload an image
uploaded = files.upload()

# Get the uploaded file name
filename = list(uploaded.keys())[0]

# Read the image
img = cv2.imread(filename)

# Check if image is loaded
if img is None:
    print("Error: Unable to load the image.")
else:
    # Convert image to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Detect faces
    faces = face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(30, 30)
    )
```